Software Requirement Specification (SRS)

VECS Android Application — VERSION 1.0 MINIMUM VIABLE SCOPE

Project Title:	VECS Android App
Lead Engineer:	AV Shayan
Target Architecture:	Native Android (Kotlin / Jetpack Compose) & Firmware Edge Client
Document Version:	v1.0.0 (Strict V1 Scope Blueprint)

1. Executive Project Summary

This requirements document establishes the absolute minimum viable scope for Version 1 of the VECS Companion Application platform. The primary objective of V1 is to validate remote end-to-end data pipelines between the vehicle's TCU (Telematics Control Unit), a cloud infrastructure broker, and a native mobile interface. Complex sub-features like navigation mapping and local proximity Bluetooth triggers are explicitly deferred to optimize development velocity and ensure telemetry connection stability.

2. Streamlined Cloud Data Pipeline (MQTT Backbone)

Version 1 implements an exclusively cloud-based topology over an online MQTT broker. The bike's TCU acts as an edge node publishing telemetric payload packets over cellular network layers. The Android client acts as a dedicated subscriber/publisher client to handle real-time UI state tracking and command distribution.

3. Application Feature Requirements Matrix (Version 1)

The scope of the active user interface and hardware pipeline is limited strictly to the following three components:

Feature Module	Functional Specification	Protocol Base
1. Range and SOC Indicator	Reads active State-of-Charge (SoC) percentages and remaining range calculations (km). Displays data at the top of the app canvas with a linear status bar that updates dynamically when telemetry payload frames land.	MQTT Cloud (Sub)
2. Ping My Bike Feature	Provides a single prominent touch target button in the app grid layout. When pressed, it instantly publishes a short command string to a designated control topic, prompting the bike's BCM to activate horn and flash headlights.	MQTT Cloud (Pub)
3. Static Vehicle Photo	Renders a high-resolution graphical asset or profile picture of the target e-bike layout inside an asymmetrical card container. For Version 1, this	Local Asset (Static)

asset is bundled statically inside the app bundle's direct local drawables folder.

4. Production & UI Constraints

4.1 User Interface & Styling Architecture

- The interface uses an asymmetrical grid structure featuring large typographical readability for vehicle status parameters, making it glanceable in real-world use cases.
- Built natively using a pure Jetpack Compose codebase structured in rows and columns without legacy layout file wrappers.

> **V1 Scope Boundary Clause:** Local Bluetooth LE (BLE) scanners, background proximity tracking, navigation tracking maps, and historical trip logs are completely omitted from the active V1 codebase. They will be added in subsequent phases after validating basic cellular telemetry and remote ping actions.